

Pipelined RISC-V CPU Verification

Rohit Gayali, vwh2up

1 CPU Results

At the end of their simulations, the single-cycle and pipelined CPUs had the same register and RAM values. The 32 register values and first 32 RAM values at the end of the simulations are in a yellow box to the left of the screenshot. The registers are listed first, starting with x0. The RAM values are listed similarly underneath the register values. Figure 1 on page 2 is a screenshot of the ModelSim Wave Results for the single-cycle CPU. Figure 2 on page 2 is a screenshot of the ModelSim Wave Results for the pipelined CPU. The single-cycle CPU took about 18,650 nanoseconds to complete execution, while the pipelined CPU took about 25,250 nanoseconds. The screenshots are in 4K resolution, so you can zoom in for higher clarity.

2 Register Forwarding

The Pipelined CPU can forward register results from the memory or writeback stages to the execute stage to mitigate data hazards. For example, the instruction `add x2, x1, x1` at address 0x10, produces a data hazard since it requires the result of the previous instruction `addi x1, x0, 1` at address 0x0C. To prevent execution delay, the CPU will forward the result of the instruction at address 0x10 from the memory stage to the ALU A and B operands in the execute stage. Figure 3 on page 3 shows a screenshot of the ModelSim Wave window showing the relevant values in the CPU demonstrating the register forward.

3 Pipeline Stall

The Pipelined CPU should stall the fetch and decode stages before register forwarding to mitigate data hazards produced by load instructions. Load instructions produce a result at the end of the Memory stage, so instructions

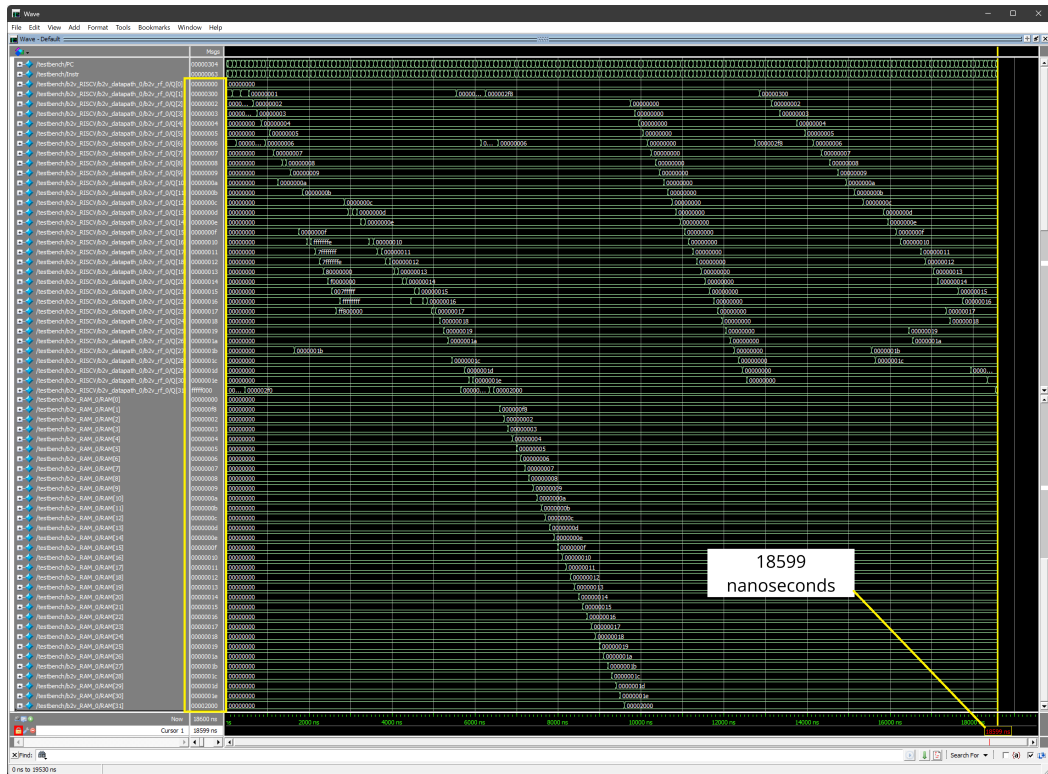


Figure 1: ModelSim Wave Results for the Single-Cycle CPU

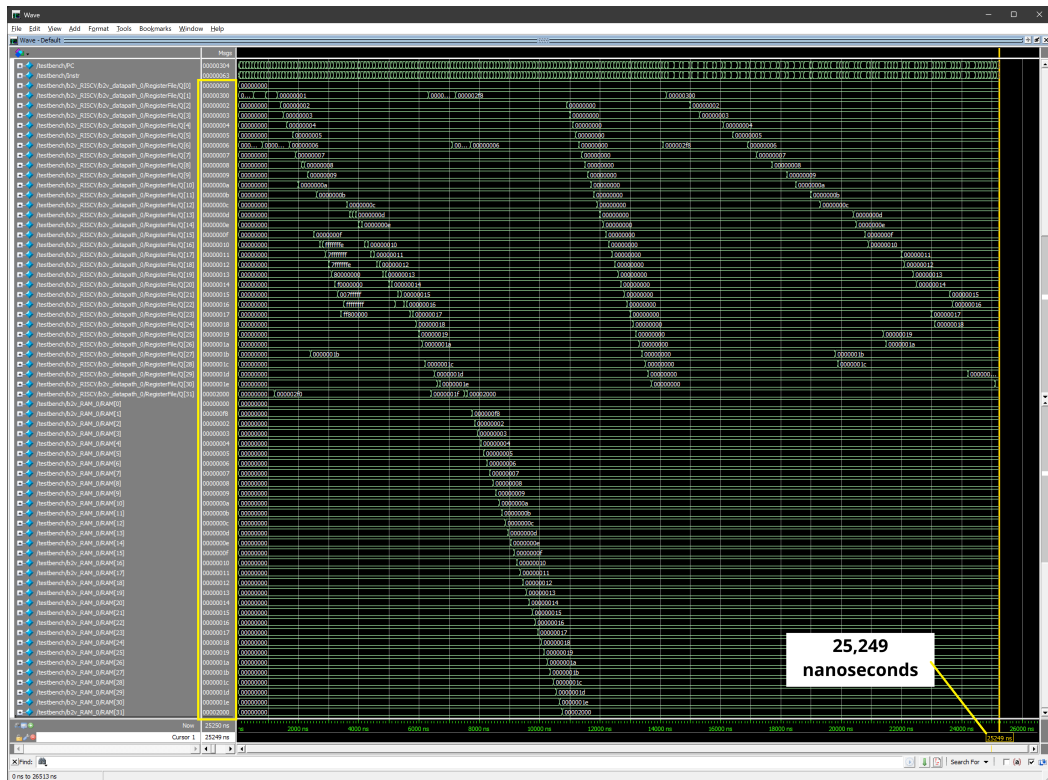


Figure 2: ModelSim Wave Results for the Pipelined CPU

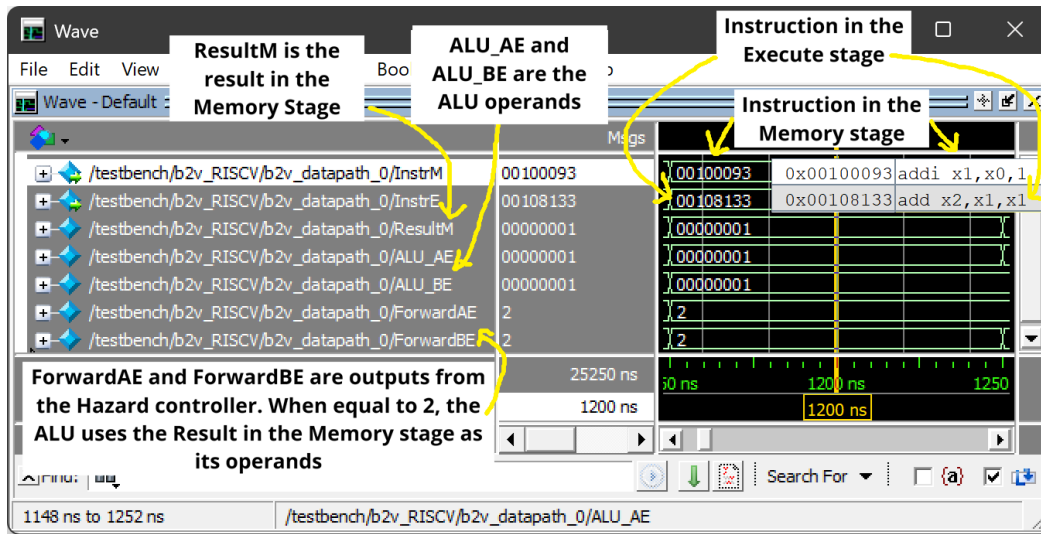


Figure 3: Pipelined CPU Hazard: Register Forwarding

in the fetch and decode stages must be stalled before forwarding the result. For example, the instruction `beq x2, x0, X2` at address `0x1F0` requires a register forward from the previous load byte instruction `lb x2, 9` at address `0x1EC`. Therefore, while the load byte instruction is in the execute stage, the instructions in the fetch and decode stages will be stalled. This ensures that the result of the load byte instruction in the memory stage can be appropriately forwarded to the next instruction in its execute stage. Figure 4 on page 4 shows a screenshot of the ModelSim Wave window showing the relevant signals in the CPU demonstrating a fetch and decode stall.

4 Pipeline Flush

The Pipelined CPU should flush instructions in the decode and execute stages to mitigate control hazards. Control hazards occur when branch and jump instructions are being executed. In addition, instructions in the execute stage should be flushed during a data hazard. For example, the instruction `jalr x1, x6, 0x2E8` at address `0x4` produces a control hazard. In response, the decode and execute stages are flushed with no op instructions until a new instruction reaches that stage. Figure 5 on page 4 shows a screenshot of the ModelSim Wave window showing the relevant signals in the CPU demonstrating a flush.

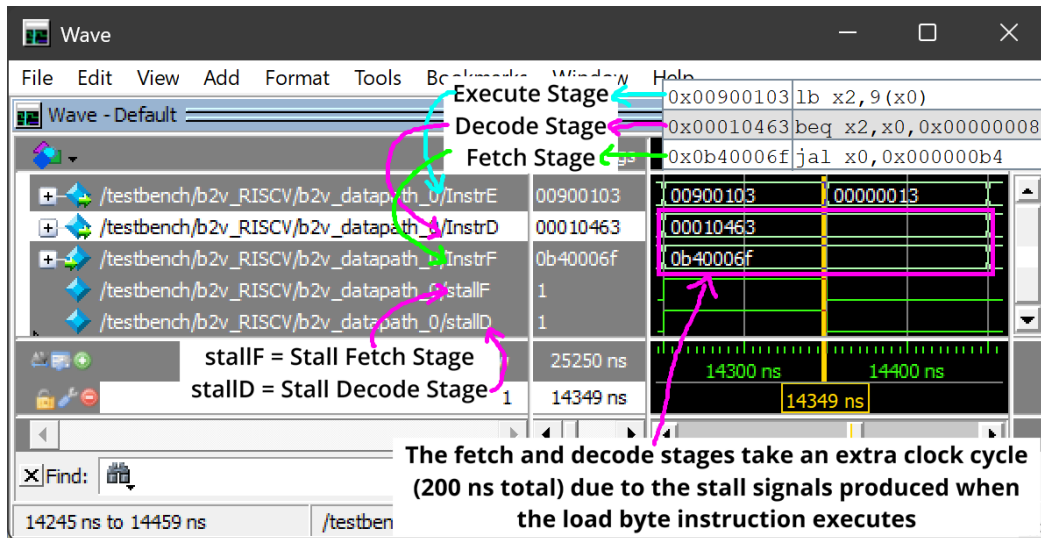


Figure 4: Pipelined CPU Data Hazard and Load Instruction Causes a Stall

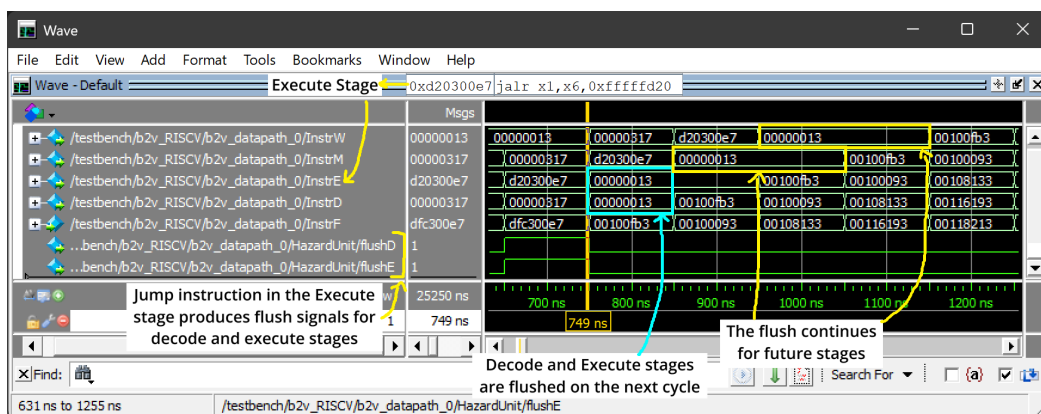


Figure 5: Pipelined CPU Control Hazard Causes a Flush